

Java Recursion Cheat Sheet

This cheat sheet covers the basics of recursion in Java with simple examples.

🌀 Basic Recursion

Recursion is a method where the solution to a problem depends on solutions to smaller instances of the same problem.

Structure:

1. Base Case - condition to stop recursion.
2. Recursive Case - function calls itself with a smaller input.

🔑 Factorial Example

```
public int factorial(int n) {
    if (n == 0) return 1;        // Base case
    return n * factorial(n - 1); // Recursive call
}
```

🔑 Fibonacci Example

```
public int fibonacci(int n) {
    if (n <= 1) return n;        // Base cases: fib(0)=0, fib(1)=1
    return fibonacci(n - 1) + fibonacci(n - 2); // Recursive call
}
```

🔑 Sum of Array

```
public int sumArray(int[] arr, int n) {
    if (n <= 0) return 0;
    return sumArray(arr, n - 1) + arr[n - 1];
}
```

🔑 Reverse String

```
public String reverse(String str) {
    if (str.isEmpty()) return str;
```

```
    return reverse(str.substring(1)) + str.charAt(0);
}
```

÷ HCF / LCM / GCD Examples

🔑 GCD (Greatest Common Divisor)

```
public int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

🔑 LCM (Least Common Multiple)

```
public int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

🔑 HCF (Highest Common Factor)

```
// HCF is another name for GCD
public int hcf(int a, int b) {
    if (b == 0) return a;
    return hcf(b, a % b);
}
```

☑ Best Practices

- Always define a base case to avoid infinite recursion.
- Ensure each recursive call moves closer to the base case.
- Prefer iteration if recursion depth is too high (risk of `StackOverflowError`).